Documento Técnico: Sistema de Gestão Logística

1. Introdução

Este documento detalha a arquitetura técnica, o design do banco de dados, as tecnologias recomendadas e um roadmap de implementação para um sistema de gestão logística web. O objetivo é criar uma aplicação robusta, escalável e de fácil utilização, acessível em múltiplos dispositivos (desktop e mobile) dentro de uma rede local, para otimizar as operações de localização de materiais, controle de estoque, gestão de serviços de iluminação pública e agendamento de despachos.

2. Requisitos Funcionais e Não Funcionais

2.1. Requisitos Funcionais

- **Finder (Localizador de Materiais):** Permite a busca eficiente de materiais por código ou descrição em um catálogo de aproximadamente 30.000 itens, retornando sua localização específica.
- **Gestão de Estoque:**
  - Registro de entrada de materiais (devoluções de obras).
  - Controle de localização e quantidade de materiais.
  - Integração para registro de entrada no ERP da empresa.
- **Gestão de Iluminação Pública:**
  - Importação e processamento de planilhas com múltiplas abas (26 bairros).
  - Consolidação e roteirização dos pontos com problemas em 6 rotas distintas.
  - Geração de relatórios para impressão e despacho aos profissionais.
- **Agenda de Despachos:**
  - Interface visual de duas colunas: cabeçalhos de despacho (data, cliente, OS, pedidos) à esquerda e detalhes dos materiais do pedido selecionado à direita.
  - Suporte a múltiplos pedidos por OS e repetição de clientes/OS em diferentes datas.
  - Notificações e lembretes para programação de despachos.
- **Gestão de Materiais e Clientes:**
  - Importação de listas de materiais e clientes via CSV/Excel.

    - Processamento de solicitações de materiais via entrada manual ou importação de PDFs.
- **Gestão de Usuários:**
    - Criação e gerenciamento de contas de usuário.
    - Definição de perfis de acesso (ex: administrador, operador de estoque, despachante).

2.2. Requisitos Não Funcionais

- **Performance:** Respostas rápidas para buscas (Finder) e processamento de dados (importações).
- **Disponibilidade:** Operação contínua na rede local.
- **Escalabilidade:** Capacidade de lidar com o crescimento do volume de dados e usuários.
- **Usabilidade:** Interface intuitiva e responsiva para desktop e dispositivos móveis.
- **Segurança:** Autenticação e autorização de usuários para proteger dados sensíveis.
- **Manutenibilidade:** Código limpo, modular e bem documentado.

3. Arquitetura da Solução

A solução proposta segue uma arquitetura cliente-servidor, com um backend robusto para processamento de dados e lógica de negócios, e um frontend responsivo para a interação do usuário. A comunicação entre frontend e backend será via API RESTful.

3.1. Componentes Principais

- **Frontend (Cliente Web):** Interface de usuário construída com tecnologias modernas para garantir responsividade e uma experiência fluida em diferentes dispositivos.
- **Backend (Servidor de Aplicação):** Responsável por toda a lógica de negócios, gerenciamento de dados, autenticação, autorização e integração com sistemas externos (ERP, processamento de arquivos).
- **Banco de Dados:** Armazenamento persistente de todos os dados da aplicação.

3.2. Diagrama de Arquitetura (Conceitual)

mermaid

Failed to render mermaid:

```
Parse error on line 2:
...ph TD    A[Usuário (Desktop/Mobile)] --
----------------------^
Expecting 'SQE', 'DOUBLECIRCLEEND', 'PE', '-)', 'STADIUMEND', 'SUBROUTINEEND',
'PIPE', 'CYLINDEREND', 'DIAMOND_STOP', 'TAGEND', 'TRAPEND', 'INVTRAPEND',
'UNICODE_TEXT', 'TEXT', 'TAGSTART', got 'PS'
```

Ask Manus to fix

4. Stack Tecnológica Recomendada

4.1. Backend

- **Linguagem/Framework:** Python com FastAPI.
  - **Justificativa:** Python é excelente para manipulação de dados (com bibliotecas como Pandas), processamento de PDFs e integração com APIs. FastAPI oferece alta performance, validação de dados automática e documentação de API interativa (Swagger UI), o que acelera o desenvolvimento e a manutenção. É também uma escolha robusta para a criação de APIs RESTful.
- **Bibliotecas Chave:**
  - `SQLAlchemy`: ORM (Object-Relational Mapper) para interação com o banco de dados, abstraindo a complexidade do SQL.
  - `Pandas`: Para manipulação e processamento de dados de planilhas (CSV/Excel).
  - `PyMuPDF` (Fitz): Para extração de texto e dados de arquivos PDF.
  - `python-multipart`: Para lidar com upload de arquivos (planilhas, PDFs).
  - `Passlib`: Para hashing seguro de senhas de usuários.
  - `python-jose`: Para implementação de JWT (JSON Web Tokens) para autenticação.

4.2. Frontend

- **Framework:** React (com Vite para desenvolvimento rápido).
  - **Justificativa:** React é uma biblioteca JavaScript amplamente utilizada para construir interfaces de usuário interativas e reativas. Sua abordagem baseada em componentes facilita a modularização e reutilização de código. Vite proporciona um ambiente de desenvolvimento extremamente rápido.
- **Estilização:** Tailwind CSS.

    - **Justificativa:** Tailwind CSS é um framework CSS utilitário que permite construir designs personalizados rapidamente, sem sair do HTML. É altamente configurável e excelente para criar interfaces responsivas que se adaptam bem a diferentes tamanhos de tela (desktop e mobile).
- **Gerenciamento de Estado:** React Context API ou Redux Toolkit (para aplicações maiores).
- **Roteamento:** React Router.

4.3. Banco de Dados

- **Sistema de Gerenciamento de Banco de Dados (SGBD):** PostgreSQL.
    - **Justificativa:** PostgreSQL é um SGBD relacional de código aberto, robusto, confiável e com recursos avançados. É ideal para lidar com grandes volumes de dados (30.000 itens), oferece excelente performance para consultas complexas e suporta indexação avançada, crucial para a funcionalidade do Finder. Além disso, é amplamente utilizado em ambientes de produção e possui uma comunidade ativa.

5. Design do Banco de Dados (Esquema Conceitual)

O design do banco de dados será focado na normalização para garantir a integridade dos dados e otimizar as consultas. Abaixo, um esquema conceitual das principais tabelas:

5.1. Tabela `Usuarios`

Armazena informações dos usuários do sistema.

Campo	Tipo de Dado	Restrições	Descrição
`id`	`INTEGER`	`PRIMARY KEY`, `AUTO_INCREMENT`	Identificador único do usuário
`nome_usuario`	`VARCHAR(50)`	`UNIQUE`, `NOT NULL`	Nome de usuário para login
`email`	`VARCHAR(100)`	`UNIQUE`, `NOT NULL`	Endereço de e-mail do usuário
`senha_hash`	`VARCHAR(255)`	`NOT NULL`	Hash da senha do usuário
`perfil`	`VARCHAR(20)`	`NOT NULL`	Perfil de acesso (ex: 'admin', 'estoque', 'despachante')

`ativo`	`BOOLEAN`	`DEFAULT TRUE`	Indica se o usuário está ativo

5.2. Tabela `Materiais`

Armazena o catálogo de materiais, incluindo os 30.000 itens.

Campo	Tipo de Dado	Restrições	Descrição
`id`	`INTEGER`	`PRIMARY KEY`, `AUTO_INCREMENT`	Identificador único do material
`codigo`	`VARCHAR(50)`	`UNIQUE`, `NOT NULL`	Código do material (para Finder)
`descricao`	`TEXT`	`NOT NULL`	Descrição detalhada do material (para Finder)
`localizacao`	`VARCHAR(100)`	`NOT NULL`	Localização física no estoque
`unidade_medida`	`VARCHAR(10)`		Unidade de medida (ex: 'un', 'm', 'kg')

5.3. Tabela `Estoque`

Registra o movimento e a quantidade atual de materiais em estoque.

Campo	Tipo de Dado	Restrições	Descrição
`id`	`INTEGER`	`PRIMARY KEY`, `AUTO_INCREMENT`	Identificador único do registro de estoque
`material_id`	`INTEGER`	`FOREIGN KEY`	Referência à tabela `Materiais`
`quantidade`	`NUMERIC(10,2)`	`NOT NULL`	Quantidade atual em estoque
`data_atualizacao`	`TIMESTAMP`	`DEFAULT CURRENT_TIMESTAMP`	Última atualização do registro

`tipo_movimento`	`VARCHAR(20)`		Tipo de movimento (ex: 'entrada', 'saida')
`referencia_movimento`	`VARCHAR(100)`		Referência do movimento (ex: OS, Pedido)

5.4. Tabela `Clientes`

Armazena informações dos clientes.

Campo	Tipo de Dado	Restrições	Descrição
`id`	`INTEGER`	`PRIMARY KEY`, `AUTO_INCREMENT`	Identificador único do cliente
`nome`	`VARCHAR(100)`	`NOT NULL`	Nome do cliente
`cnpj_cpf`	`VARCHAR(20)`	`UNIQUE`	CNPJ ou CPF do cliente
`contato`	`VARCHAR(100)`		Nome do contato
`telefone`	`VARCHAR(20)`		Telefone de contato
`email`	`VARCHAR(100)`		E-mail de contato

5.5. Tabela `OrdensServico`

Armazena as Ordens de Serviço (OS).

Campo	Tipo de Dado	Restrições	Descrição
`id`	`INTEGER`	`PRIMARY KEY`, `AUTO_INCREMENT`	Identificador único da OS
`cliente_id`	`INTEGER`	`FOREIGN KEY`	Referência à tabela `Clientes`
`numero_os`	`VARCHAR(50)`	`UNIQUE`, `NOT NULL`	Número da Ordem de Serviço
`data_criacao`	`DATE`	`NOT NULL`	Data de criação da OS

descricao	TEXT		Descrição da OS

5.6. Tabela Pedidos

Armazena os pedidos de materiais associados a uma OS.

Campo	Tipo de Dado	Restrições	Descrição
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Identificador único do pedido
os_id	INTEGER	FOREIGN KEY	Referência à tabela OrdensServico
numero_pedido	VARCHAR(50)	UNIQUE, NOT NULL	Número do pedido de material
data_solicitacao	DATE	NOT NULL	Data de solicitação do pedido
data_despacho_prevista	DATE		Data prevista para o despacho
status	VARCHAR(20)	DEFAULT 'pendente'	Status do pedido (ex: 'pendente', 'despachado')

5.7. Tabela ItensPedido

Detalha os materiais de cada pedido.

Campo	Tipo de Dado	Restrições	Descrição
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Identificador único do item do pedido
pedido_id	INTEGER	FOREIGN KEY	Referência à tabela Pedidos
material_id	INTEGER	FOREIGN KEY	Referência à tabela Materiais
quantidade_solicitada	NUMERIC(10,2)	NOT NULL	Quantidade de material solicitada

`quantidade_despachada`	`NUMERIC(10,2)`	`DEFAULT 0`	Quantidade de material despachada

5.8. Tabela `IluminacaoPublicaPontos`

Armazena os pontos de iluminação pública com problemas.

Campo	Tipo de Dado	Restrições	Descrição
`id`	`INTEGER`	`PRIMARY KEY`, `AUTO_INCREMENT`	Identificador único do ponto
`bairro`	`VARCHAR(50)`	`NOT NULL`	Bairro onde o problema foi reportado
`descricao_problema`	`TEXT`	`NOT NULL`	Descrição do problema
`data_registro`	`DATE`	`NOT NULL`	Data de registro do problema
`rota`	`VARCHAR(10)`		Rota atribuída para o reparo
`status`	`VARCHAR(20)`	`DEFAULT 'pendente'`	Status do reparo (ex: 'pendente', 'concluido')

6. Roadmap de Implementação

Este roadmap sugere uma abordagem iterativa para o desenvolvimento do sistema, começando pelos módulos mais críticos e construindo sobre eles.

Fase 1: Configuração e Base de Dados

1. **Configuração do Ambiente:** Instalação de Python, Node.js, PostgreSQL e ferramentas de desenvolvimento (VS Code, Git).
2. **Configuração do Banco de Dados:** Criação do banco de dados PostgreSQL e definição das tabelas (`Usuarios`, `Materiais`, `Clientes`).

3. **Backend Básico:** Configuração do FastAPI, conexão com o PostgreSQL via SQLAlchemy e criação de endpoints CRUD básicos para `Usuarios`, `Materiais` e `Clientes`.
4. **Autenticação e Autorização:** Implementação de login de usuário com JWT e controle de acesso baseado em perfis.

Fase 2: Módulo Finder e Gestão de Materiais

1. **Importação de Materiais:** Desenvolvimento de funcionalidade para importar a lista de 30.000 itens de materiais (CSV/Excel) para a tabela `Materiais`.
2. **Finder:** Implementação da lógica de busca por código ou descrição no backend e criação da interface de usuário no frontend.
3. **Gestão de Clientes:** Desenvolvimento da funcionalidade de importação de clientes (CSV/Excel) e interface de gerenciamento.

Fase 3: Gestão de Estoque e Ordens de Serviço

1. **Entrada de Estoque:** Desenvolvimento da funcionalidade de registro de entrada de materiais (devoluções) e atualização da tabela `Estoque`.
2. **Gestão de OS e Pedidos:** Criação de endpoints e interfaces para gerenciar `OrdensServico`, `Pedidos` e `ItensPedido`.
3. **Importação de Solicitações:** Implementação da funcionalidade de importação de solicitações de materiais via PDF e entrada manual.

Fase 4: Agenda de Despachos

1. **Backend da Agenda:** Desenvolvimento de endpoints para listar despachos por data, cliente, OS e pedidos, e para detalhar os itens de um pedido específico.
2. **Frontend da Agenda:** Construção da interface de duas colunas, com a lista de despachos à esquerda e os detalhes dos materiais à direita, com funcionalidade de clique para exibir os detalhes.

Fase 5: Módulo de Iluminação Pública

1. **Importação de Planilhas:** Desenvolvimento da funcionalidade de importação e processamento das planilhas de iluminação pública (26 abas).
2. **Roteirização:** Implementação da lógica para consolidar e atribuir os pontos com problemas às 6 rotas distintas.
3. **Geração de Relatórios:** Criação de funcionalidade para gerar relatórios imprimíveis das rotas para os profissionais.

Fase 6: Refinamentos e Implantação Local

1. **Testes:** Realização de testes unitários, de integração e de aceitação para garantir a qualidade do sistema.
2. **Otimização de Performance:** Otimização de consultas SQL e lógica de processamento, especialmente para o Finder e importações.
3. **Documentação:** Elaboração de documentação técnica e de usuário.
4. **Implantação Local:** Configuração do ambiente de produção em um servidor local (PC), incluindo Docker para facilitar a implantação e gerenciamento dos serviços (PostgreSQL, FastAPI, Nginx para servir o frontend).

7. Considerações Finais

Este documento fornece uma base sólida para o desenvolvimento do sistema de gestão logística. A escolha de tecnologias modernas e comprovadas, juntamente com um design de banco de dados bem estruturado, garantirá uma aplicação performática, segura e de fácil manutenção. A abordagem iterativa do roadmap permitirá a entrega de valor contínuo e a adaptação a possíveis mudanças nos requisitos ao longo do projeto.

Autor: Manus AI

Data: 25 de Abril de 2026